Applied Soft Computing Journal 93 (2020) 106386

Contents lists available at ScienceDirect

Applied Soft Computing Journal

journal homepage: www.elsevier.com/locate/asoc

Unsupervised phase learning and extraction from quasiperiodic multidimensional time-series data

Prayook Jatesiktat [a,*], Guan Ming Lim [b], Christopher Wee Keong Kuah [c], Wei Tech Ang [a,b]

[a] Rehabilitation Research Institute of Singapore, Nanyang Technological University, 11 Mandalay Road, 308232, Singapore
[b] School of Mechanical and Aerospace Engineering, Nanyang Technological University, N3-01a-01, 50 Nanyang Avenue, 639798, Singapore
[c] Rehabilitation Centre, Centre for Advanced Rehabilitation Therapeutics, Tan Tock Seng Hospital, 308440, Singapore

ARTICLE INFO

Article history:
Received 23 February 2019
Received in revised form 4 May 2020
Accepted 4 May 2020
Available online 8 May 2020

Keywords:
Phase
Unsupervised learning
Neural network
Signal processing

ABSTRACT

Automatic phase extraction is commonly done by digital markers or handcrafted feature detection that are designed specifically for one particular type of signal. A few more generalized ways are to apply Hilbert transform or complex wavelet convolution. However, they are limited to single-dimensional signals, and they require the input window to cover multiple signal periods to extract a smooth phase sequence. In this work, we propose a learning-based phase extraction method for multi-dimensional signals, consuming only unlabeled signal examples for training. A neural network architecture is designed to map a window of signal directly to a phase value under a key constraint that encourages two consecutive phase values to progress forward. The concept has been generalized to a broad range of applications with an adjustable input window, a flexible phase progression penalty design, and better training stability. The proposed method consistently outperforms the complex wavelet convolution in three synthetic signals by reducing the phase extraction error by 63% while shortening the size of the observation window by over 90%. An experiment with an augmented electrocardiogram (ECG) signal also produces a clean phase sequence with an input window that is over thirty times smaller. A qualitative test on eleven-dimensional kinematic signals from three repetitive upper limb movements has shown that the method can also capture the fluctuation in phase progression. With the flexibility, the labelless training, the accuracy, and the short observation window, this technique has a potential to be used in a variety of applications that need instantaneous phase extraction.
© 2020 Elsevier B.V. All rights reserved.

1. Introduction

Periodic phenomena that repeat themselves at regular intervals are commonly found in nature and things we measure every day. Our heartbeats, breathing, locomotion, and exercise routine are a few examples. However, the periodicity of these signals is not always perfect. The signal can be contaminated with noise. The frequency or phase progression can vary over time. The ability to extract phase information from such inconsistent periodic signals will provide a fundamental tool that can enhance the automation in time-series analysis applications such as analysis of movements of living things or analysis of biomedical signals.

There are a few common techniques to assign a phase value to each sample in the signal. The simplest way is to manually assign a known phase to distinct key points of every period and fill in the gaps with linear [1] or non-linear interpolation. However, this method is not scalable with a large data size.

To automate the process, specific feature detection algorithms can be designed to assign some pre-known phases associated with the features. For example, a specially handcrafted feature is used in QRS complex detection of electrocardiogram (ECG) signal [2]. In human gait analysis, the signature of a heel strike can be used to identify the transition point from a swing phase to a stance phase [3].

Phase could also be assigned explicitly during a data collection with help from digital markers such as a mechanical switch or capacitive sensors on the object for human movement analysis [4,5]. With these digital markers, the method does not need to use waveform for segmentation and phase assignment.

Nonetheless, those features and digital markers are often designed specifically for a particular type of signal, and they need to be redesigned for every new type of signal. In addition, they need interpolation to fill the gaps between those key points. One non-linear technique that can be used for the interpolation is dynamic time warping (DTW). DTW has become a common technique in human movement research to align multiple trials of a movement [6]. Still, the need for a manually segmented template in DTW prevents the whole procedure from being fully automated.

* Corresponding author.
E-mail addresses: prayook001@e.ntu.edu.sg (P. Jatesiktat), guanming001@e.ntu.edu.sg (G.M. Lim), christopher_kuah@ttsh.com.sg (C.W.K. Kuah), wtang@ntu.edu.sg (W.T. Ang).

https://doi.org/10.1016/j.asoc.2020.106386
1568-4946/© 2020 Elsevier B.V. All rights reserved.

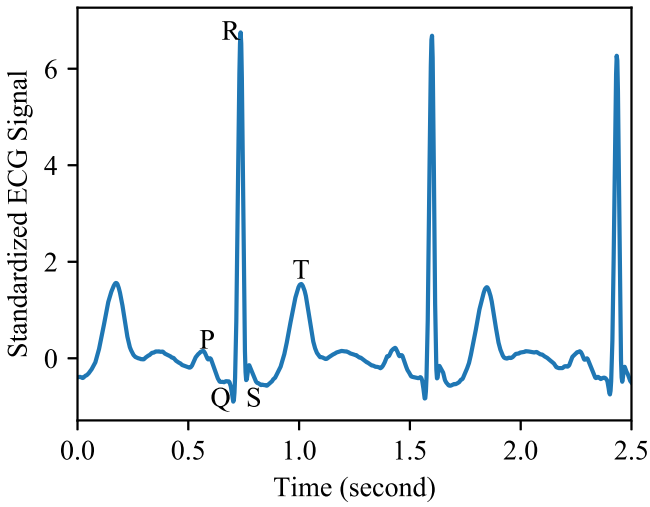


Fig. 1. An example of ECG signal which consists of multiple peaks from P wave, QRS complex, and T wave in one heartbeat. This characteristic causes difficulties in feature-free phase extraction methods such as Hilbert transform or complex wavelet transform.

If the generalization of the procedure is desired, there are two common ways of phase extraction. The first way is Hilbert transform [7] which assumes that the input signal itself is just the real part of a complex signal. The method can recover the imaginary part of the signal from the real part. Then, the instantaneous phase can be calculated from the angle in the complex coordinate [8]. The second way is to convolute the input signal with a complex Morlet wavelet [9]. The phases of the complex time series result can be used to estimate the phases of the signal. This complex convolution can be thought of as a parametric alternative of the Hilbert transform where the center frequency and the bandwidth of the wavelet can be chosen [10].

However, one limitation of the Hilbert transform is the assumption that the signal should be similar to a sinusoidal pattern for the method to have a good phase estimation. If the signal rises and falls multiple times in one period, it can disturb the extracted phase sequence. An example of those signals is ECG which contains a P wave, a QRS complex, and a T wave in one heartbeat (Fig. 1). For the complex Morlet wavelet convolution method, the effective length of its convolution filter is normally configured to cover at least 3–5 signal periods or more for the phase extraction to work. This large filter size can reduce the responsiveness in a time-sensitive application. In addition, both methods are designed for single-channel time-series data. If there are multiple channels from the same source, there is no obvious way to combine all those channels to extract just one phase sequence.

So far, the use of learning methods to enhance the automation of phase extraction has not been explored much. Unlike other classification problems with discrete output, one difficulty of learning the phase extraction is the continuity of the phase which makes a manual labeling process tedious and inaccurate. Because of this fact, we believe that unsupervised learning is a more practical way to solve this phase extraction problem.

Our objective is to develop an unsupervised learning technique that can learn to extract phase from quasiperiodic multidimensional time-series data without any pre-segmentation or any phase label in the training data. In our early work, we propose a repetitive signal modeling method using two autoencoders which can be used to extract phase [11]. However, this method is slow because it needs a large number of neural-network feed-forwarding iterations just to extract phase at a time instance.

To speed up the phase extraction, another strategy is proposed to let a neural network learn a direct mapping from the signal to the phase output for repetitive movements of humans' rehabilitative exercise and rats' gait [12]. This article extends and improves the method to be generalized to a broader range of applications. The key improvements from the previous paper are summarized as follows:

1. The input to the neural network is changed from a single sample to an adjustable window of consecutive samples. This change allows the user to balance between the responsiveness of the application (by selecting a narrower input window) and the stability of estimation (by selecting a wider input window).
2. The "phase progression penalty profile" is designed to replace the "clockwise penalty" term in the cost function used in the training stage. This new term provides the network with greater flexibility to handle more challenging scenarios such as ECG data.
3. Quantitative comparisons between the proposed method, Hilbert transform, and complex Morlet wavelet convolution are performed.

2. Method

2.1. Method overview

The method consists of a learning step followed by an inference step. First, a neural network is trained with one or more sequences of high-dimensional time-series data with each sequence containing more than one period of the target signal. Fragmentation of the signal to multiple short sequences should be avoided for better training stability. As these training sequences do not contain any phase label, a special cost function is designed to force the network to organize itself and learn the concept of phase.

Every pair of consecutive sampling windows will be mapped by the neural network to two consecutive phase values. Intuitively, a proper mapping should align these two consecutive phase values in a gradually progressive way. Collectively, with a large enough number of pairs in the training data, the network will eventually be able to find a usable phase mapping to all samples in the training data under the constraints of the cost function. After the learning is completed, a phase can be inferred by simply feed-forwarding the movement data through the trained network.

2.2. Neural network architecture

The network architecture is shown in Fig. 2. The input layer consists of two sections which are the signal section and its heading direction section.

The first section is a window of $2w+1$ consecutive samples of the multi-channel signal. A channel in this context means a one-dimensional time-series sequence of an actual recorded signal or any additional features extracted from the same source. Each sample of the signal is a K-dimensional vector.

The second section is a window of $2w+1$ consecutive samples of normalized vectors of K slopes. Each vector is called *signal heading direction*. Because the signal is a sequence of K-dimensional vectors, the signal heading direction at each sample is also a K-dimensional vector. The calculation of the signal heading direction is described in Section 2.3.

In some applications, the signal and its trend at two distinct phase values could be very similar if the observing window is too narrow. This ambiguity can be solved by expanding the input

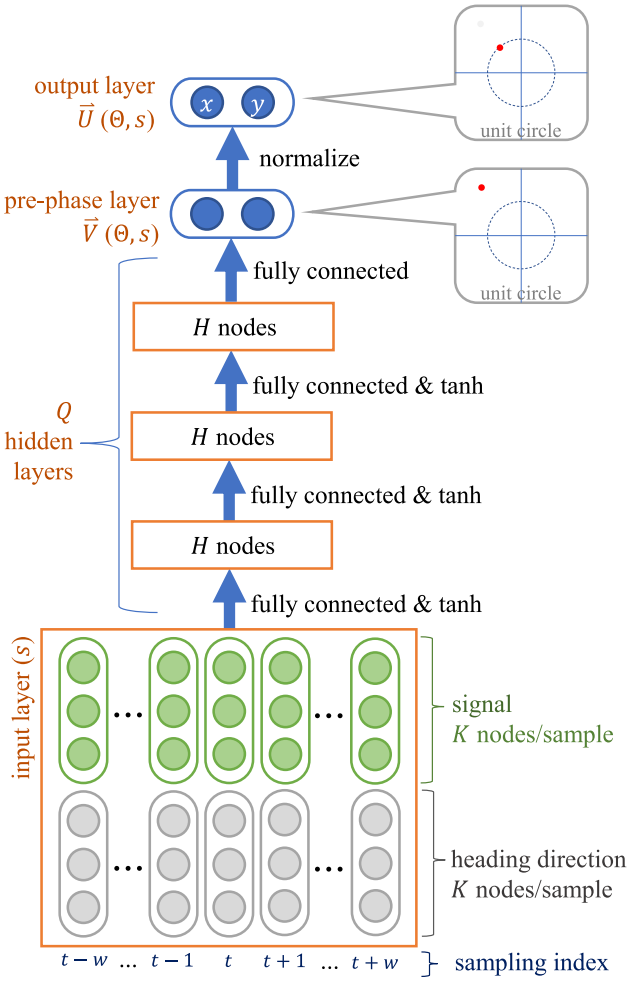


Fig. 2. Neural network architecture for phase extraction. The output from the feed-forwarding process will always land on the unit circle which represents the instantaneous phase of the input window.

window to the length that the phase values can be clearly differentiated. However, expanding the window will always cost some responsiveness in the application as it has to wait longer for more future samples. Therefore, the input layer of the neural network is designed to be adjustable in size with the parameter w. The input layer of our previous design [12] is equivalent to having $w = 0$ in this new design. With $w \geq 1$, more clues from neighboring samples are given to the input layers to reduce the ambiguity in phase mapping to solve more complex cases. However, with a large number of dimensions in the signal, the ambiguity is likely to be reduced. This high-dimensional case usually allows the parameter w to be smaller.

The network has Q fully connected hidden layers with hyperbolic tangent activation function. Each hidden layer contains H hidden nodes. All nodes in the last hidden layer are fully connected to two nodes in the pre-phase layer with linear activation function. Then, the result in the pre-phase layer is normalized to a 2D unit vector at the output layer.

The feed-forwarding function $\vec{U}(\Theta, s) = [x\ y]^\top$ can be seen as a transformation from the $2K(2w + 1)$-dimensional input layer (s) to a 2D point on a unit circle (x, y). All the trainable weights and biases in the network are represented by the symbol Θ. The symbol s represents all the $2w + 1$ samples of the standardized signal and the heading direction that are packed into the input layer of the network. This 2D representation of phase is chosen to avoid the discontinuous wrapping point (at 0 and 2π) in the 1D phase representation that usually confuses the neural network. However, to simplify plots in this article, the 2D phase is converted to the 1D phase value in a range of $[0, 2\pi)$. The conversion from the 2D form (x, y) to the 1D form (ϕ) is done by

$$\phi = \arctan 2(y, x)\%(2\pi). \tag{1}$$

Since the *arctan2* operator has its range of $(-\pi, \pi]$, the modulo operator (%) is used to map it to a positive range of $[0, 2\pi)$.

2.3. Training data preparation

The training data is first standardized so that all K dimensions (from the signal section) have the mean of zero and the standard deviation of one.

At each standardized sample $\vec{p}_t$, the $2r + 1$ neighboring samples $\vec{p}_{t-r}, \ldots, \vec{p}_{t+r}$ are used to calculate the K-dimensional signal heading direction $\vec{h}_t$.

For each dimension, a linear regression is performed on $2r+1$ data samples to extract one slope. Then, K independent slopes from all K dimensions are combined and normalized to a unit vector $\vec{h}_t$.

As the periodic signal can be seen as a repetitive trajectory that moves in a K-dimensional space with a mixture of arbitrary speeds, the normalization helps the phase learning to be invariant to the speed of the movement in the K-dimensional signal space. Therefore, the parameter r should be selected so that the trend of the signal can be captured within $2r + 1$ consecutive samples. If the signal contains a lot of noise, the parameter r may have to be larger to make the heading direction more robust.

Every two consecutive input windows are packed together as a pair $\langle \bar{s}_i, \hat{s}_i \rangle$ where $\bar{s}_i$ is from the earlier window and $\hat{s}_i$ is from the later window with a one-sample shift. If the first input window $\bar{s}_i$ contains $\vec{p}_{t-w}, \ldots, \vec{p}_{t+w}$ and $\vec{h}_{t-w}, \ldots, \vec{h}_{t+w}$, the second input window $\hat{s}_i$ will contain $\vec{p}_{t+1-w}, \ldots, \vec{p}_{t+1+w}$ and $\vec{h}_{t+1-w}, \ldots, \vec{h}_{t+1+w}$.

2.4. Cost function

In the training, all the weights and biases parameters Θ are searched to minimize a cost function defined as

$$\begin{aligned} F(\Theta) = & \frac{1}{N}\sum_{i=1}^{N} \mathcal{P}(\arctan 2(\bar{x}_i\hat{y}_i - \hat{x}_i\bar{y}_i\ ,\ \bar{x}_i\hat{x}_i + \bar{y}_i\hat{y}_i)) \\ & + \alpha\left[\left(\frac{1}{N}\sum_{i=1}^{N}\bar{x}_i\right)^2 + \left(\frac{1}{N}\sum_{i=1}^{N}\bar{y}_i\right)^2\right] \\ & + \frac{\gamma}{N}\sum_{i=1}^{N}\frac{1}{\sigma\sqrt{2\pi}}\exp\left(-\frac{|\vec{V}(\Theta, \bar{s}_i)|^2}{2\sigma^2}\right) \end{aligned} \tag{2}$$

where $\vec{U}(\Theta, \bar{s}_i) = [\bar{x}_i\ \bar{y}_i]^\top$ and $\vec{U}(\Theta, \hat{s}_i) = [\hat{x}_i\ \hat{y}_i]^\top$ are the feed-forwarding calculations from data pair i. The symbol N is the number of input pairs in the training data. The function $\vec{V}(\Theta, \bar{s}_i)$ is a 2D activation vector from the pre-phase layer.

The first *phase progression penalty* term from Eq. (2) is designed to force every pair of consecutive inputs to have a proper amount of phase progression. The calculation of this penalty from a pair of consecutive inputs can be illustrated in Fig. 3.

Consider a data pair i which is transformed to the first output point $(\bar{x}_i\bar{y}_i)$ and the second output point $(\hat{x}_i\hat{y}_i)$ on the unit circle. The penalty for this pair of data depends on the amount of the phase progression from the first to the second output. To make the calculation of the progression automatically differentiable, their cross product and dot product are computed. Given that

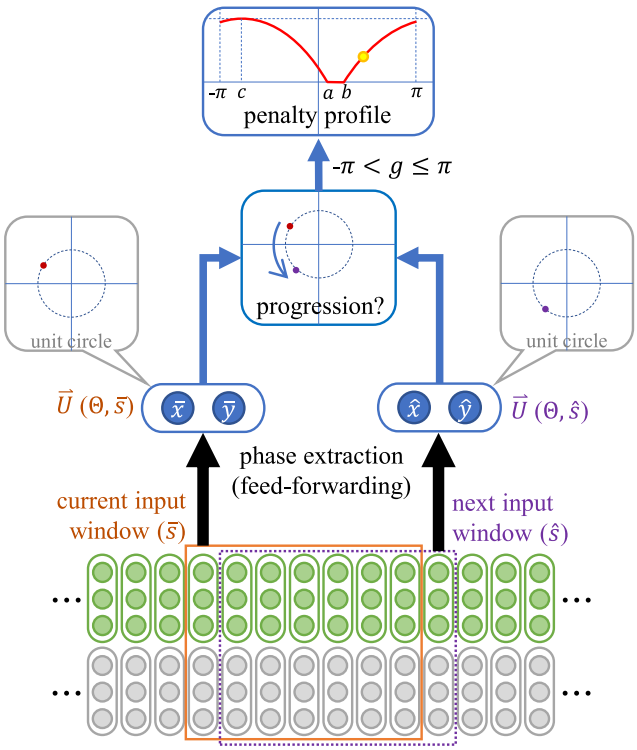


Fig. 3. Calculation of phase progression penalty from one pair of consecutive input windows.

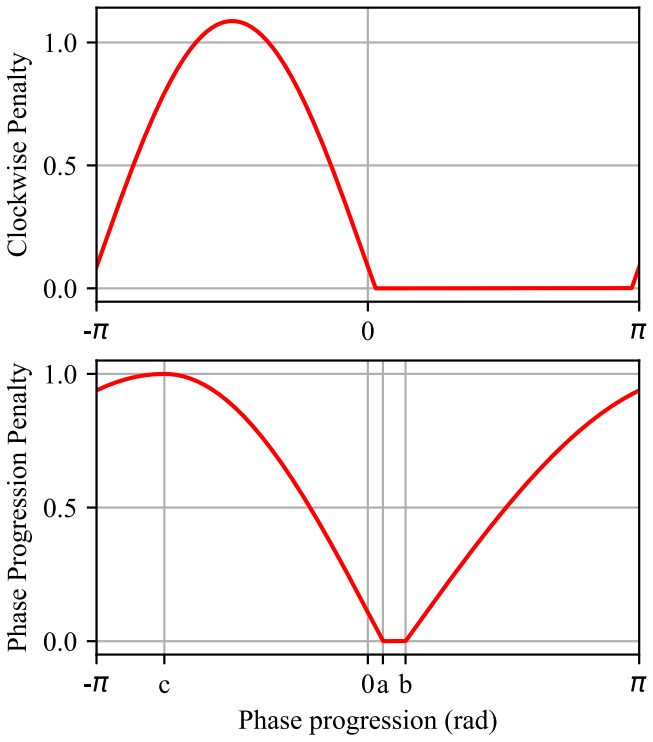


Fig. 4. A comparison between the clockwise penalty profile from our previous work (top) and the phase progression penalty profile used in this work (bottom). The key differences are adjustable positions of the peak (c) and the valley (a, b) that introduce an additional penalty for positive phase progression.

$g \in (-\pi, \pi]$ is the unknown phase progression in the unit of radian, the cross product $(\bar{x}_i\hat{y}_i - \hat{x}_i\bar{y}_i)$ will be equal to $\sin(g)$ and the dot product $(\bar{x}_i\hat{x}_i + \bar{y}_i\hat{y}_i)$ will be equal to $\cos(g)$. Then, g can be recovered using *arctan2* function as

$$g = \arctan 2(sin(g), cos(g)) \tag{3}$$

$$g = \arctan 2(\bar{x}_i\hat{y}_i - \hat{x}_i\bar{y}_i\ ,\ \bar{x}_i\hat{x}_i + \bar{y}_i\hat{y}_i). \tag{4}$$

The function $\mathcal{P} : (-\pi, \pi] \mapsto \mathbb{R}$ is a penalty profile that maps a phase progression to a penalty value. The function is defined as

$$\mathcal{P}(g) = \begin{cases} \cos(\frac{\pi}{2}(g-c)/(2\pi+c-b)) & \text{if } -\pi < g \le c \\ \cos(\frac{\pi}{2}(g-c)/(a-c)) & \text{if } c < g \le a \\ 0 & \text{if } a < g \le b \\ \cos(\frac{\pi}{2}(g-c-2\pi)/(2\pi+c-b)) & \text{if } b < g \le \pi \end{cases} \tag{5}$$

where c, a and b are adjustable parameters under conditions $-\pi < c < a < b < \pi$. This penalty profile is visualized in Fig. 4. This function is designed to be continuous throughout the domain $(-\pi, \pi]$ including the warping point between π and $-\pi$.

This *phase progression penalty* has been evolved from *clockwise penalty* proposed in our previous work [12]. For comparison, the profile of *clockwise penalty* (Fig. 4) is defined as

$$\mathcal{P}_{clockwise}(g) = \max\big(0, -\sin(g) + \epsilon\big) \quad \text{with} \quad \epsilon > 0. \tag{6}$$

The main feature of the clockwise penalty is to penalize the network whenever it produces a negative phase progression ($-\pi < g < 0$). In addition, lifting the whole negative sine curve up a little bit with ϵ generates negative slopes in the extra area where the phase progression is slightly above zero. This slope encourages any two consecutive samples to leave a gap of positive progression between them in the mapped phase space. However, we realize that it is not necessary to have a positive phase progression in every sample as some applications may allow zero phase progression or even a little bit of negative phase progression in some situations. This flexibility is not supported by simply adjusting ϵ to zero or below because that configuration will create flat slope at the $g = 0$ and $g = \pi$ which allows all the phase values to collapse to only two opposite locations on the unit circle as shown in Fig. 5. Furthermore, the large flat area on the positive side of the clockwise penalty allows the phase to progress arbitrarily fast in one step. With a complex signal shape, the optimization can easily converge to a solution that all the progressions become multiple times of the expected speed and the clockwise penalty cannot penalize the network to diverge away from the unwanted solution. All these limitations in the clockwise penalty are addressed in the new design of phase progression penalty profile.

The side-by-side comparison between the previous and the new phase progression penalty profile is illustrated in Fig. 4. Similarly, the new profile still penalizes the negative side of the phase progression with some adjustments for the peak and the valley. One important change is the additional penalty on the positive progression (when $g > 0$). For a general signal, if it is sampled with a sufficient sampling rate to obtain the complete waveform, the phase will not largely progress between consecutive samples. Therefore, the proper range of phase progression should be in a small range above zero, and the phase progression that is too large should be penalized.

In the newly proposed penalty profile, the zone without penalty is narrowed down to an adjustable range from a to b. These parameters can be adjusted according to the sampling rate and the estimated period length of the target signal. If the prior knowledge about the signal is known, the parameter a can be set as the minimum phase progression of the target signal in one sample pair. Setting a as a positive value will encourage the network to avoid the *phase collapse* situation (Fig. 5). In general, this parameter can be set as zero.

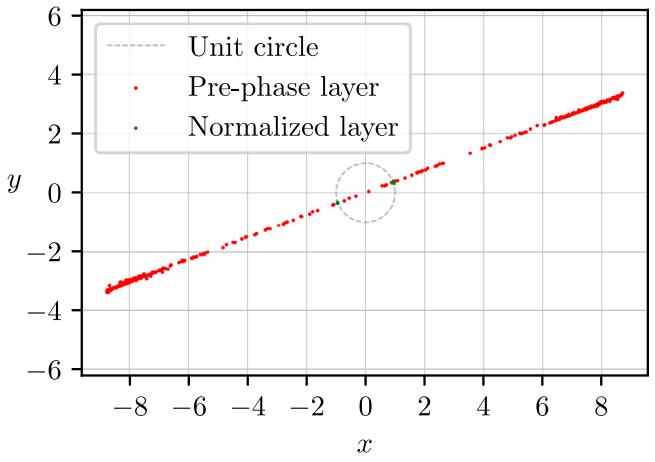


Fig. 5. An example of the *phase collapse* situation in which the trained network maps the training data along a diagonal line of the unit circle in the pre-phase layer (red) and does not diverge from this state. Thus, output phase values collapse into two opposite clusters (green).

On the other hand, the parameter b can be set as the maximum phase progression of the target signal in one sample pair. Because the area above b of this penalty profile impedes the phase progression not to be excessively fast in one step, it automatically reduces the chance of producing more than one round of phase progression within a single period. This effect is especially beneficial for the experiment with the ECG signal as discussed in an experiment in Section 4.5 that compares the training stability between the clockwise penalty profile and the new phase progression penalty profile.

The purpose of the parameter c is to destroy the stability of the unwanted *phase collapse* situation (Fig. 5) especially when $a \leq 0$. Intuitively, the penalty should be the highest at the most negative phase progression of $-\pi$ or $c = -\pi$. However, that condition unintentionally creates a stable point for the *phase collapse* state by providing a near-zero gradient around the phase progression of $-\pi$. This stability can be removed by shifting the parameter c away from $-\pi$ (such as $c = -0.75\pi$). This shift creates an obvious slope around the phase progression of $-\pi$ which encourages any pair of consecutive samples that is mapped to the opposite side of the unit circle to move closer and finally close the loop.

The second term of Eq. (2) is called *distribution penalty*. It calculates the centroid of the pre-phase outputs from all samples in the training data and measures the squared distance from the origin (*i.e.*, the center of the unit circle). This term reflects how well the outputs are distributed on the unit circle. In the case that the outputs are not evenly distributed, the centroid will shift away from the center, and the network will be penalized. However, the *distribution penalty* term does not work perfectly by itself. It is possible for the neural network to transform all the training data into one straight line on the pre-phase layer and create two roughly equal clusters of phase on the opposite side of the unit circle (Fig. 5). This mapping behavior perfectly satisfies the *distribution penalty* term without reaching their objective of distributing all the mapping throughout the unit circle. This problem is solved by adjusting the phase progression penalty profile parameter c to be away from $-\pi$ as being discussed in the previous paragraph.

The last *singularity penalty* term of Eq. (2) is designed to prevent the pre-phase layer from staying too close to the origin. Due to the normalization in the last layer, this is the area where the phase output can be unstable especially with noisy inputs. The penalty can be visualized as a 2D Gaussian function with the highest penalty at the origin in the pre-phase space. The penalty reduces with more distance from the origin. The parameter σ controls the width (e.g. standard deviation) of the Gaussian curve. Larger the σ, farther the points will be pushed away from the origin in the pre-phase layer. In general, σ can be set to 1.0. This setup will produce the standard normal curve which keeps the points at least 3 units away from the origin.

The weight parameter α and γ are two hyperparameters that are introduced to reduce the aggressiveness of the distribution penalty and the singularity penalty when needed. In some situations when the optimization is trapped in an undesired local minimum, there is a need to disturb the distribution penalty term and the singularity penalty term to increase for a short moment to allow the phase progression penalty term to approach a new minimum. If α and γ is too high, the optimizer might not be able to find a way out of the trap. Some trials and errors may be required to get these weights right. In general, they can be set to 1.0 or below as the most important term is the phase progression penalty.

2.5. Initialization

All the weights are initialized randomly from a Gaussian distribution with zero mean and standard deviation of $1/\sqrt{n}$ where n is the number of nodes in the previous layer of that weight. All the biases are initialized as zeros.

2.6. Optimization (training)

The cost function (Eq. (2)) is implemented on TensorFlow [13] to utilize its automatic differentiation capability. Adam optimization algorithm [14] with *learning_rate* $= 0.001$, *beta1* $= 0.9$, *beta2* $= 0.999$, and *epsilon* $= 10^{-8}$ is used to minimize the cost function for 5000 iterations in all experiments. These Adam optimization parameters are set according to the default recommendation by Kingma and Ba [14].

3. Experiment 1: 2D synthetic data

This section validates the capability of the method to learn from periodic signals that form 2D loops in different shapes. The use of synthetic data allows us to have the ground truth phase to compare with the extracted phase. Such a comparison cannot be done quantitatively with any actual signals.

3.1. Synthetic signal generator

Three shapes of signal with different characteristics are chosen. Each signal is governed by two signal generators $G_x(t)$ and $G_y(t)$ with an equal period of 2π *i.e.*, $G(t+2\pi) = G(t)$. Fig. 6 shows the shape and the waveform of each signal with and without disturbance.

Heart shape. This is an uncommon loop with both smooth and sharp turning points. The signal is generated by

$$G_x(t) = 1.8\sin^3(t) \tag{7}$$

$$G_y(t) = 1.3\cos(t) - 0.5cos(2t) - 0.2cos(3t) - 0.1cos(4t). \tag{8}$$

Bow shape. The loop is twisted, and it contains one crossing point in the phase-space plot. To properly extract phase from this shape, the trained network must create a transformation that untwists the trajectory into a non-crossing loop. The signal is generated by

$$G_x(t) = 1.4\cos(t) \tag{9}$$

$$G_y(t) = 2.9\sin(t)\cos(t). \tag{10}$$

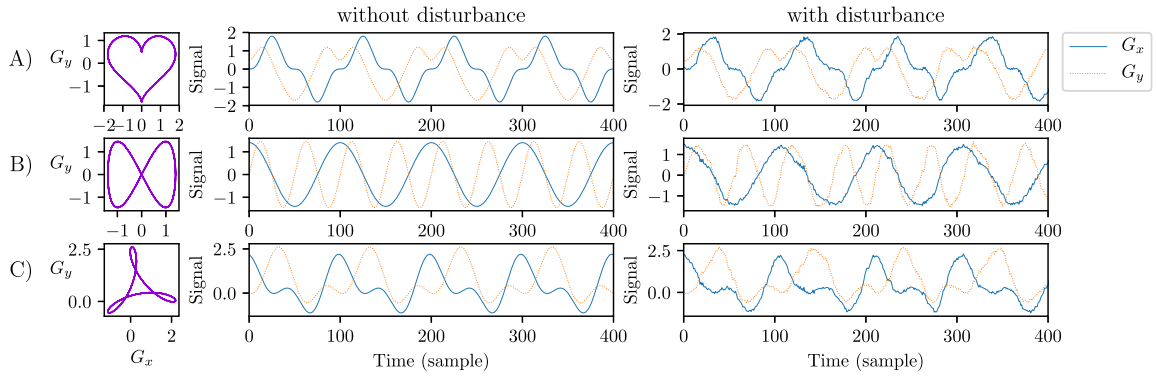


Fig. 6. Three synthetic signals used in the experiment. **Row A**: Heart shape. **Row B**: Bow shape. **Row C**: Acrobat shape. The first two columns illustrate the signals with constant phase progression and without noise. The last column illustrates the signals with fluctuated phase progression and with additional signal noise.

Acrobat shape. This shape is selected to explore the case of multiple crossing points in a loop. In addition, the waveforms of both channels go up and down twice in one period which makes them challenging for other techniques such as Hilbert transform. The signal is generated by

$$G_x(t) = 1.6\cos(t)(\sin(0.5t + 0.5) + \cos(1.5t + 0.5)) \quad (11)$$

$$G_y(t) = 1.6\sin(t)(\sin(0.5t + 0.5) - \cos(1.5t + 0.5)). \quad (12)$$

3.2. Signal disturbance

To simulate uncertainties, the generated data are disturbed in both temporal and spatial ways as illustrated in Fig. 6.

The time sampling starts at $t = 0$ and the progression in each step is sampled from a uniform distribution ranging from 0 to $4\pi/100$. The signal is expected to be sampled around 100 times per period. This random progression creates inconsistency in every step of the phase progression. Then, the sampled data in both x and y channels are added with Gaussian noise with zero mean and 0.05 standard deviation. These disturbed data are ready for the process described in Section 2.

3.3. Training configuration

For each shape, a sequence of 15,000 samples from the disturbed signal is used as the training data. The neural network contains 5 hidden layers with 30 nodes in each layer ($Q = 5$, $H = 30$). Other parameters are $r = 7$, $w = 3$, $a = 0$, $b = 4\pi/180$, $c = -0.75\pi$, $\alpha = 1$, $\gamma = 1$, and $\sigma = 1$. The training is done for 5000 iterations.

3.4. Result: Training data distribution in pre-phase layer

For all three shapes, all the training data are mapped to one clean loop surrounding the origin in the pre-phase layer (Fig. 7A). For the bow shape and the acrobat shape, all the crossing points are unfolded within 500 and 1500 iterations respectively. Fig. 7B shows that each part of the signal is mapped consistently to a narrow range of phase value. The results from every training iteration are available in the supplementary videos.

3.5. Result: Phase extraction from unseen test data

The trained networks are used to extract phase from a newly generated sequence (15,000 samples) with the same disturbances. Some examples of the phase extraction results are shown in Fig. 8.

Because the data is synthetic, the ground-truth phase value at any sample of the signal is known as $t\%(2\pi)$. A perfect phase extraction should provide a constant difference between the phase output and $t\%(2\pi)$. Therefore, the variation of the phase difference such as range and standard deviation can be used to evaluate the quality of the phase extraction.

Given a collection of the differences between an extracted phase (ϕ_j) and its corresponding ground-truth phase ($t_j\%2\pi$) as $d_j = (\phi_j - t_j)\%2\pi$, the range and standard deviation cannot be calculated directly from d_j due to the wrapping point of the value at 0 and 2π. Instead, all the points (d_j) have to be rotated by an angle so that the warping point (0 and 2π) falls in the biggest gap between any two adjacent values (see Fig. 9). After the rotation, the range and the standard deviation can be calculated normally.

Our phase extraction provides a fairly constant difference from the ground-truth phase with standard deviations of 0.049, 0.047, 0.048 rad for the heart, the bow, and the acrobat shape respectively. In all three cases, the ranges of variation from our method are under 0.39 rad which is just about 6% of maximum possible range (2π).

3.6. Quantitative comparison with available methods

Two traditional methods available for phase extraction are the Hilbert transform and the complex wavelet convolution. Both methods are designed to be used only with one-dimensional time-series data. Therefore, to make the comparison possible, the two-dimensional synthetic signals used in this experiment have to be reduced into a one-dimensional signal. To do so, Principal Component Analysis (PCA) is used to project the 2D signal into the first principal component. Then, the result of this dimensionality reduction is used in both methods.

In the case of the Hilbert transform, we treat the reduced dimension as the real part and the imaginary part is constructed by the Hilbert transform. Then, *arctan2* function is used to extract phase from the real and the imaginary part.

In the case of complex wavelet convolution, a complex Morlet wavelet filter is constructed for each configuration according to

$$\Psi(x) = (\pi F_B)^{-0.5}\exp(2xF_C\pi\,\mathrm{i})\exp(-x^2/F_B) \quad (13)$$

given that x is evenly sampled at the data sampling rate (100 Hz) in the interval $[-B, B]$. The boundary parameter $B = P/(2F_C)$ determines the length of effective support. The center frequency F_C is set to 1 Hz to match the expected frequency of all three synthetic signals. The positive time-decay parameter F_B is set to $B^2/5$ to make the wavelet amplitude decay to a near-zero value (e^{-5}) at both ends of the effective support. The parameter P determines the length of the effective support in the unit of period. In this experiment, P is adjusted to 0.5, 1, 2, 3, and 4 periods to observe the performance at different filter sizes in comparison with the proposed method. Applying the complex wavelet convolution filter to the one-dimensional data reduced

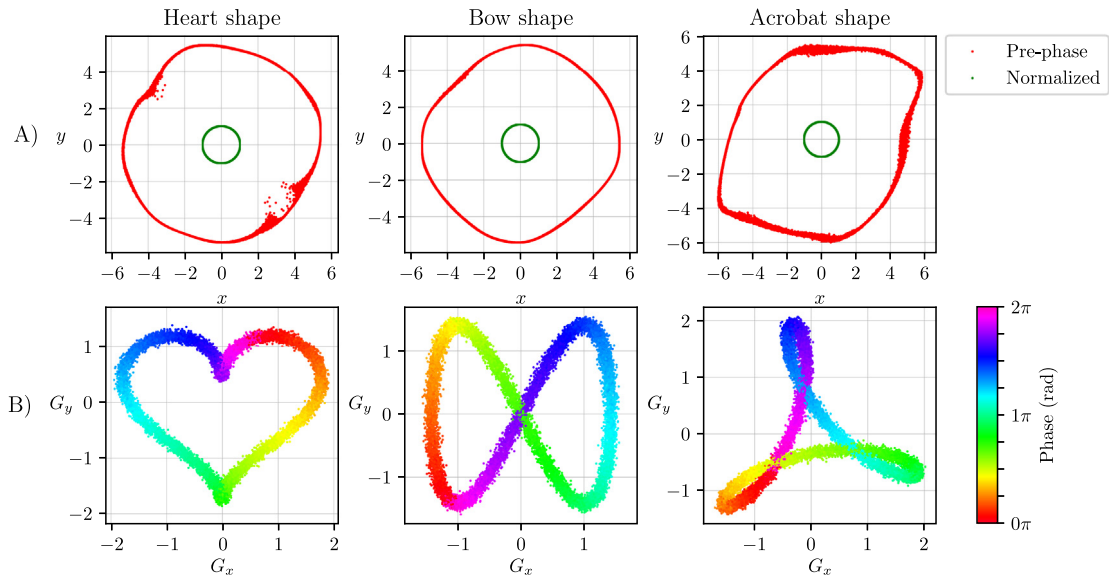


Fig. 7. Learning results from the heart, the bow, and the acrobat shape at the 5000th iteration. **Row A**: Training data distribution at the pre-phase layer and the normalized layer. **Row B**: Standardized training data in phase-space diagrams painted with the extracted phase.

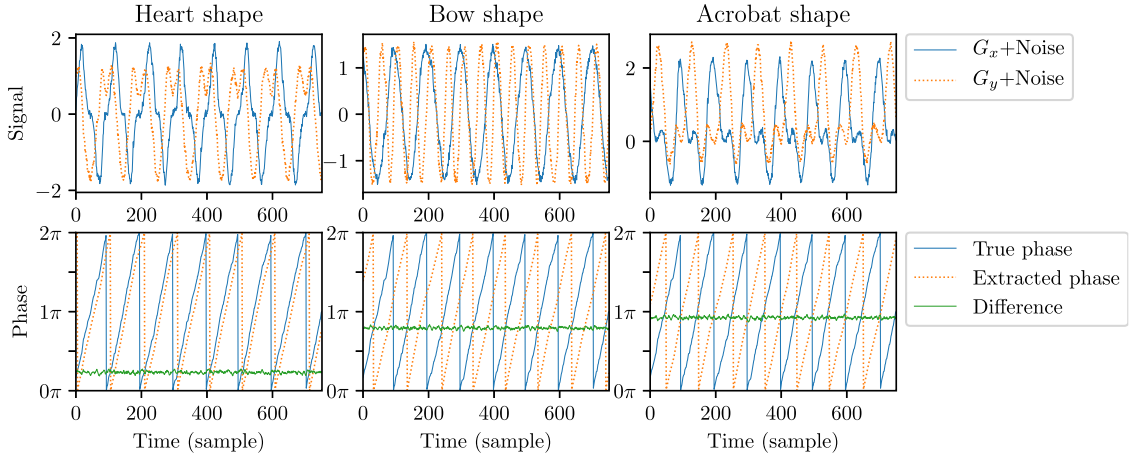


Fig. 8. Phase extraction results from the heart, the bow, and the acrobat signal. **Top row**: Disturbed signals generated for testing. **Bottom row**: Comparisons between the ground-truth phase sequences and the extracted phase sequences. A perfect phase recovery is expected to provide a sequence of constant phase difference.

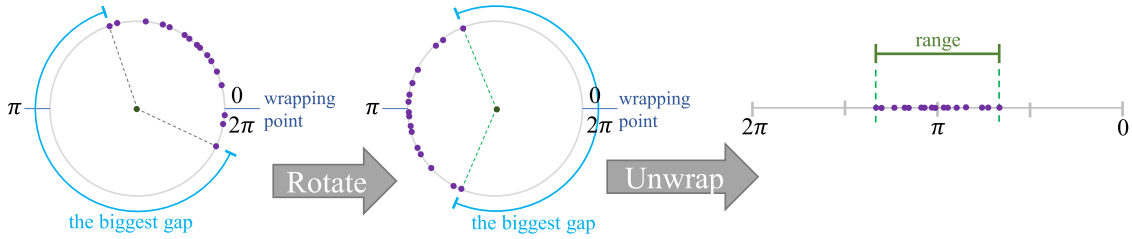


Fig. 9. Left: The collection of the differences between an extracted phase and its corresponding ground-truth phase on a unit circle. **Middle and Right:** The whole collection has to be rotated before the calculation of range and standard deviation to avoid the wrapping point between values.

from the synthetic signal will produce a complex time-series output. Then, *arctan2* function is used to extract a phase sequence from the real and the imaginary part.

The phase extraction outputs from the Hilbert transform method and all five configurations of the complex wavelet convolution method are compared to the ground-truth phase sequences to calculate ranges and standard deviations as described in Section 3.5.

The quantitative comparisons of the results are shown in Table 1. The proposed method outperforms the traditional methods in all three signals. On average, our method reduces standard deviation and range from the best configuration of the wavelet method by 63.3% and 63.1% respectively. While the best configuration of the wavelet method consumes 300 consecutive samples (about 3 signal periods) to produce one phase value, our proposed method only consumes 21 consecutive samples (with $r = 7$ and

$w = 3$) to produce one phase output with greater accuracy. This huge reduction in the size of the input window is critical for time-sensitive applications. Instead of waiting a few whole periods of signal to get enough data for an accurate phase prediction, the proposed method allows the application to respond much quicker by observing just a small portion of the signal period.

4. Experiment 2: ECG data

For single-channel data like ECG, at least one more channel should be added to make it become multi-dimensional time-series data. When the data has more than one channel, the heading direction clue will become more useful. Some possibilities of those additional dimensions could be a constantly delayed signal [2], or the imaginary output from the Hilbert transform of the original signal [8]. In this experiment, the signal is augmented using the imaginary component from the Hilbert transform.

As suggested by Pikovsky et al. the phase-space plot of the original ECG data against its Hilbert transform is not suitable for the phase extraction [8]. This problem is reproduced and demonstrated in Fig. 10. Applying *arctan2* function right after the Hilbert transform is not able to extract a phase sequence that progresses cleanly in one direction regardless of the anchor position. Because the ECG signal goes up and down multiple times in one heartbeat, the Hilbert transform will generate multiple tangled loops and no anchor point on that phase-space plot can be chosen as the center coordinate for a good phase generation. Landing the anchor point in any area will generate large portions of backward phase progression that should not occur. Landing the anchor point in some areas can generate two rounds of phase progression per heartbeat which can lead to confusion in signal interpretation.

Our method overcomes this problem by learning a transformation that unravels those tangled loops into one simpler loop that surrounds the origin before the phase extraction.

4.1. Data source

Ten segments from a subject (103) in a public ECG dataset [15] are used in this experiment. These data are recorded at 360 Hz for 10 s per segment. Nine segments are used as a training set and one segment is used for testing. The ECG data is pre-processed by three standard steps.

1. Apply a notch filter at 60 Hz to remove the frequency of alternating current.
2. Apply a first-order Butterworth low-pass filter at 40 Hz cutoff frequency.
3. Apply a median filter with the window size of one second.

Then, the second channel is added by the complex component from the Hilbert transform using Marple's algorithm [16]. These data are fed to the process described in Section 2.

4.2. Training configuration

The neural network contains 5 hidden layers with 30 nodes in each layer ($Q = 5, H = 30$). Other parameters are $r = 7$, $w = 10$, $a = 0$, $b = \pi/180$, $c = -0.75\pi$, $\alpha = 0.02$, $\gamma = 0.02$, and $\sigma = 1$. The training is run for 5000 iterations.

4.3. Result: Training data distribution in pre-phase layer

The changes in training data distribution in the pre-phase layer from the initialization to the last iteration are illustrated in Fig. 11. All the crossing points are untangled after around 2500 training iterations. The phase assignment to the training data is shown in Fig. 12. It reveals that the method could learn the complication of this ECG trajectory where three crossing points and some overlapping occur in a small region.

4.4. Result: Phase extraction from unseen data

The testing ECG segment is fed-forward to the trained network to extract phase, and the result can be seen in Fig. 10. The phase sequence from our method is much cleaner than the direct extraction after the Hilbert transform.

For comparisons, the complex wavelet convolution method is also implemented to extract phase from the real part of the pre-processed ECG data from Section 4.1. This process transforms the one-dimensional real data sequence into a complex data sequence that can be used for phase extraction. The complex Morlet wavelet is configured similarly to the one described in Section 3.6 and Eq. (13) with some changes. The evaluation of $\Psi(x)$ is done at 360 Hz to match the data sampling rate. The center frequency (F_C) is changed to 1.2 Hz to closely match the heart rate in the data. The parameter P is configured to 0.5, 1, 2, 3, and 4 periods to vary the effective support of the wavelet to 150, 300, 600, 900, and 1200 samples respectively.

The result of the complex wavelet convolution is illustrated in Fig. 13. For the complex wavelet convolution to extract phase cleanly, not only the expected central frequency of the signal has to be known but the method also requires a long enough wavelet size (e.g., the effective support of the wavelet). When the wavelet length is too short in the time domain, the filtering result will have an effect of a wide bandpass filter which includes unwanted frequency components and causes the noisy result. On the other hand, a longer wavelet filter will act as a narrower bandpass filter at the central frequency which produces a cleaner result. For this method to work well with the ECG signal, the filter size has to be around 4 times of a signal period (1200 samples) or more. However, the trained phase extraction network from our proposed method only requires about 12% of a signal period (35 samples with $w = 10$ and $r = 7$) which is over 30 times shorter than the length required by the wavelet-based method. For applications in general, this smaller input window allows an application to be more responsive because it does not have to wait until a few rounds of data period is collected before a phase calculation.

4.5. Convergence in training

To measure the improvement from switching from the clockwise penalty profile to the phase progression penalty profile. The training is repeated 100 times with the new penalty profile and 100 times with the clockwise penalty profile ($\epsilon = 0.1\pi/180$). The neural network weight initialization is random with the same 100 seeds for both configurations. All other parameters and training data are kept the same.

For the clockwise penalty, even after 50,000 training iterations the optimization always converges to an unwanted state in which the extracted phase loops two to five rounds per heartbeat. None of the 100 random initializations leads to a proper phase extraction solution. This is due to the inability of the clockwise penalty to prevent the phase from progressing too fast, and the challenging shape of the augmented ECG signal which is multiple tangling loops.

On the other hand, by using the proposed phase progression penalty profile, all of the 100 initializations converge successfully to a desired solution. The optimization takes on average 3472 iterations to converge with the maximum number of 11,074 iterations.

5. Experiment 3: Repetitive human movements

One long-term goal of this project is to apply this method in an analysis of human movement for rehabilitation purposes. Therefore, the method is also tested on human movement data.

Table 1
The variation of the difference between the extracted phase sequence and the ground-truth phase sequence of the synthetic signals (lower is better). The five configurations of the complex wavelet convolution vary the size of the convolution filters from 50 to 400 samples. The proposed method achieved the best performance both in standard deviation and range despite the smallest input window needed at 21 samples (with $r = 7$ and $w = 3$).

Method	Heart		Bow		Acrobat	
	SD (rad)	Range (rad)	SD (rad)	Range (rad)	SD (rad)	Range (rad)
Hilbert transform	0.2845	1.3357	0.9364	6.2153	0.5478	2.3022
Complex wavelet (50)	0.7467	3.6689	1.2430	6.1427	0.9238	4.5297
Complex wavelet (100)	0.4100	2.2028	0.5226	3.8588	0.4515	2.9247
Complex wavelet (200)	0.1776	1.0991	0.2442	1.6648	0.2022	1.3764
Complex wavelet (300)	0.1240	0.8561	0.1392	1.0660	0.1315	1.0658
Complex wavelet (400)	0.1413	0.9465	0.1486	1.0745	0.1438	1.0686
Proposed method	0.0494	0.3778	0.0473	0.3432	0.0483	0.3811

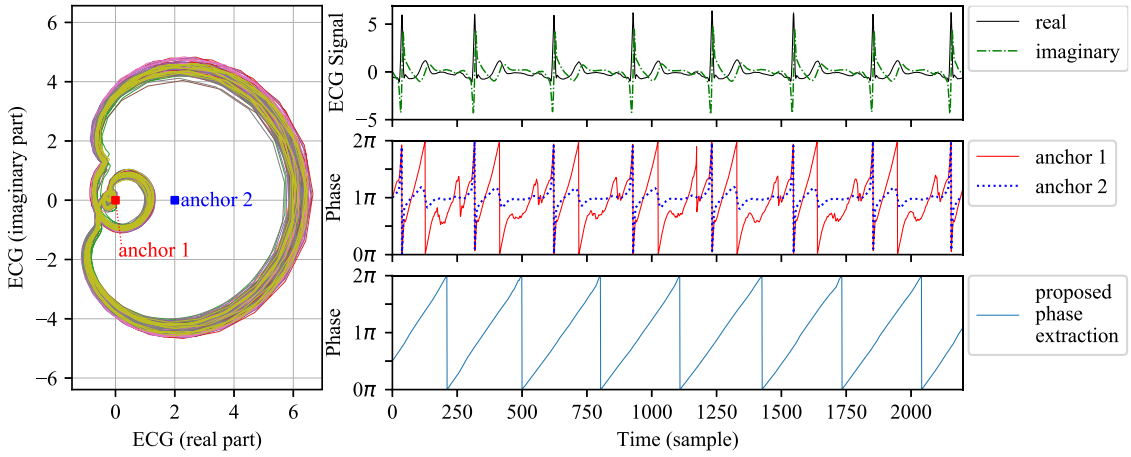


Fig. 10. Left: A plot of the real part of the ECG signal against its imaginary part augmented by Hilbert transform. **Top Right**: The testing ECG sequence with its Hilbert transform (imaginary). **Middle Right**: Phase extraction right after the Hilbert transform with two different anchor points. **Bottom Right**: The tested ECG signal is transformed by the trained network before the phase extraction.

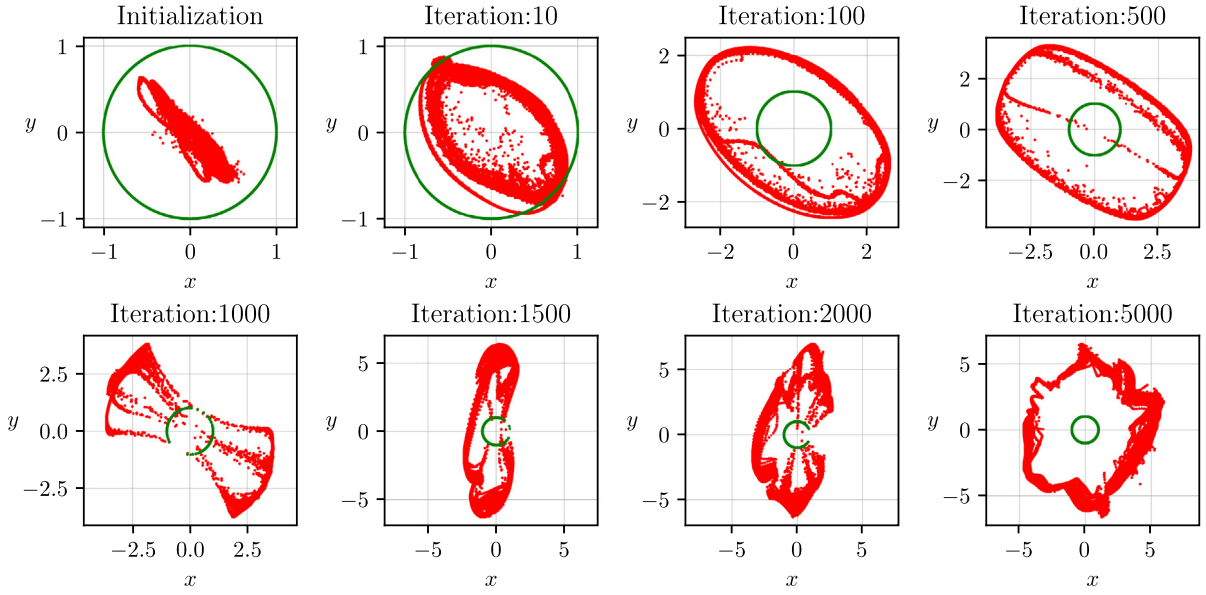


Fig. 11. Progresses in training data distribution in the pre-phase layer (red points) during the training process of ECG data from the initialization (top left) to the 5000th iteration (bottom right). The green points are their normalized results to the unit circle. The animation from every iteration is provided in the supplementary video.

5.1. Data source

Twelve healthy subjects are recruited to perform three repetitive exercises, (1) rectangular drawing (RECT), (2) hair combing (COMB), and (3) tap in a loop (TAP). For RECT, a subject slides his right hand on a table to draw a rectangle. For COMB, a subject starts with his hand on a table and lifts the hand to comb his hair and puts the hand back on the table. For TAP, a subject starts with his right hand on a table and raises the hand to touch his nose and his left shoulder and puts the hand back on the table.

Those movements are recorded at 30 frames per second with an upper-limb motion capture system that combines a Microsoft

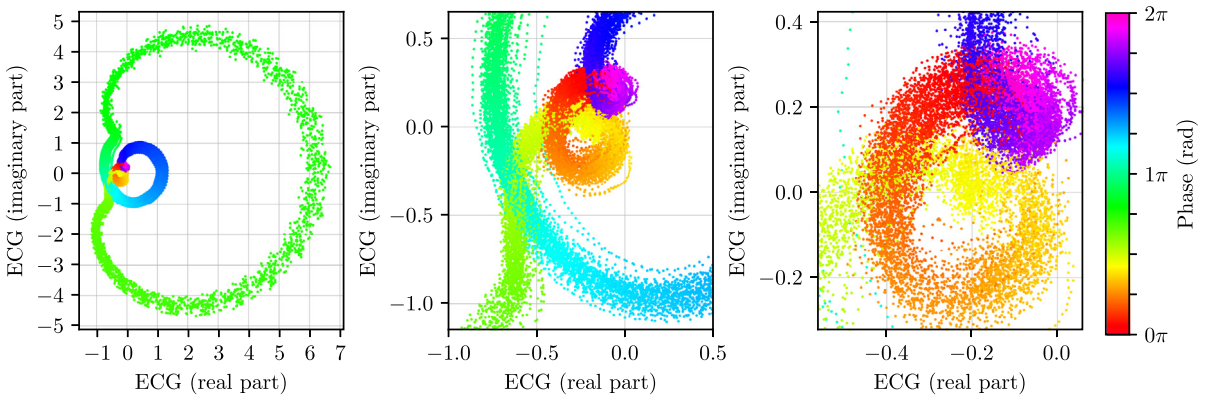


Fig. 12. **Left**: A phase-space plot of the training ECG data with an assigned phase for every sample. **Middle and Right**: The same plot being zoomed into the cluttered area of the signal where multiple crossings and overlaps occur.

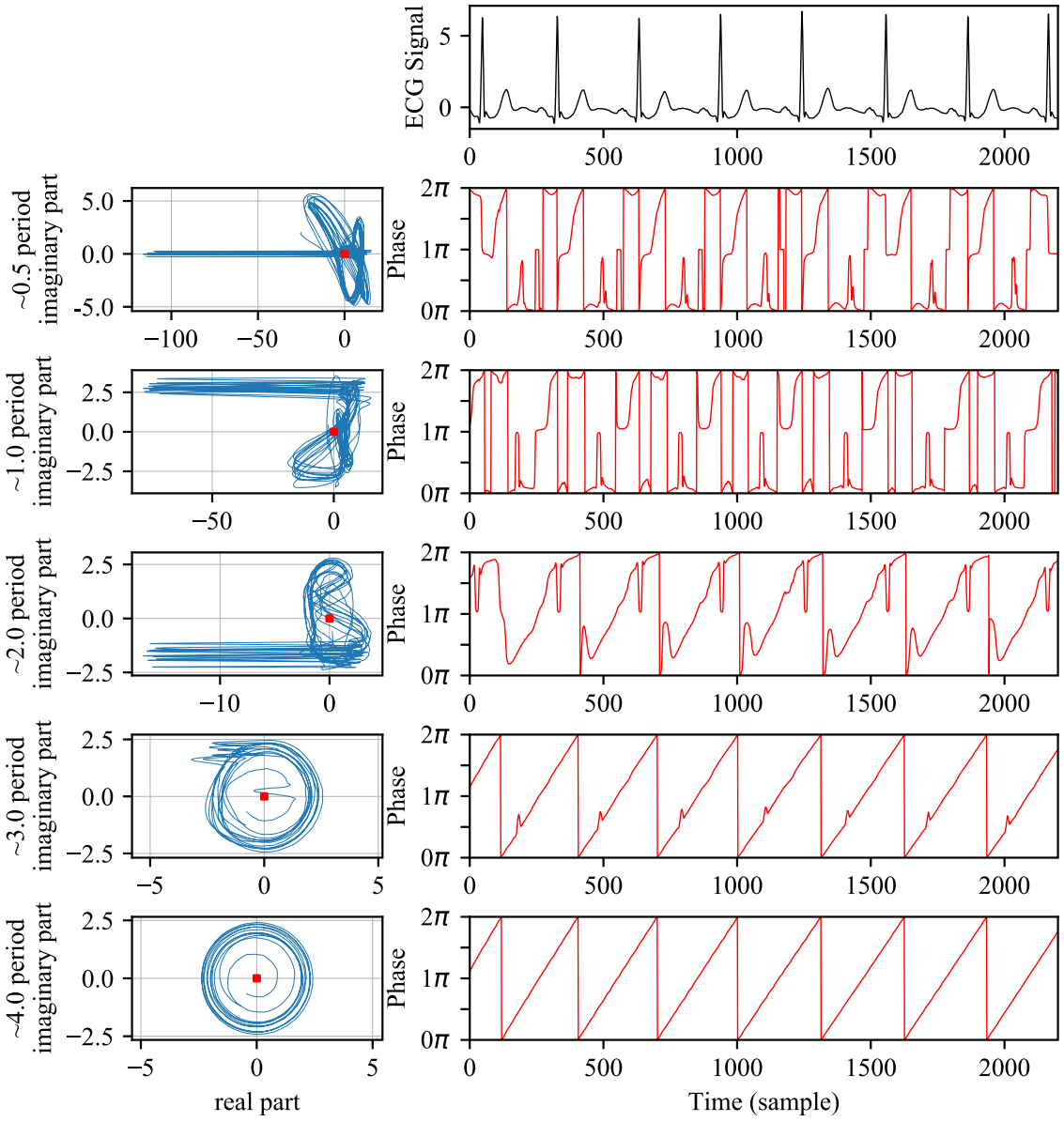


Fig. 13. Phase extraction results from the complex wavelet convolution method. The method is configured with different filter sizes which range from 150 samples (about 0.5 times of a signal period) in the second row to 1200 samples (about 4 times of a signal period) in the last row. **Top Right**: A section of the tested ECG sequence. **Left Column**: A plot of the imaginary part against the real part of the complex wavelet convolution result. **Right Column**: Direct phase extraction after the complex wavelet convolution using *arctan2* function around the origin anchor point. This method requires a relatively large input window to achieve a clean phase extraction result.

Kinect V2 and wrist-worn IMUs [17]. We record two sessions of 50 repetitions per exercise per subject. Those repetitions are done continuously without any event marker. All subjects are asked to mix their own slow, medium, and fast paces to their movements for data variations. Data from 10 subjects are used as the training set. The recorded data are fed into the training process without any segmentation. Data from the two remaining subjects are excluded for the phase extraction test.

In this experiment, we focus only on the upper torso and the right limb by extracting the following features as an eleven-dimensional vector for each frame:

- right upper-arm pointing direction (two-dimensional azimuthal equidistant projection from a three-dimensional unit vector)
- right forearm pointing direction (two-dimensional azimuthal equidistant projection from a three-dimensional unit vector)
- right forearm pronation/supination angle
- upper torso left/right turning angle
- upper torso frontal flexion
- upper torso lateral flexion
- three-dimensional wrist position

The recorded sequences of these 11 dimensions are fed to the process described in Section 2 with a few modifications to the cost function explained in Section 5.2.

5.2. Cost function modifications

With the proposed cost function (Eq. (2)), there are two problems that need to be addressed.

Firstly, the movement waveform from each subject can be significantly different due to human anthropometric variations. Sometimes during the training, the data from one or two subjects are clustered wrongly in the pre-phase layer while the data from the majority of the subjects form proper loops around the origin. As the *distribution penalty* is calculated from the centroid from all the training data, the effect of the minority of the data which distribute badly can be easily ignored. The problem can be fixed by modifying the *distribution penalty* term to calculate the same formula for each continuous data sequence and take the maximum value as the representation of the *distribution penalty* term. The term is modified to

$$\alpha \max_j \left[\left(\frac{1}{N_j} \sum_{i=q_j+1}^{q_j+N_j} \bar{x}_i \right)^2 + \left(\frac{1}{N_j} \sum_{i=q_j+1}^{q_j+N_j} \bar{y}_i \right)^2 \right] \tag{14}$$

where j is looping through each continuous data sequence, and each sequence is from index q_j+1 to q_j+N_j. By this modification, the movement record that has the worst distribution will be focused and fixed by the back-propagation until it becomes on a par with other data sequences.

Secondly, we aim to apply this technique to the application where the phase learning is done on healthy subjects and the phase extraction is applied to unhealthy subjects such as stroke patients. This means that the method should improve its generalization in order to extract phase from unseen movements that are not entirely normal. To do so, the new *marginal singularity penalty* is introduced as an addition to the cost function. Instead of using the normal input data ($\bar{s}_i$) as being used in *singularity penalty*, this new term adds some random noise before feeding it to the network. The *marginal singularity penalty* can be defined as

$$\frac{\lambda}{N} \sum_{i=1}^{N} \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{\left|\vec{V}(\Theta, \xi(\bar{s}_i, \sigma_w))\right|^2}{2\sigma^2}\right) \tag{15}$$

where the function $\xi(\bar{s}_i, \sigma_w)$ adds Gaussian noise with zero mean and σ_w standard deviation to all the dimensions of $\bar{s}_i$ followed by normalizations in all heading direction data. The added noises are randomly generated in every training iteration. The parameter λ is the weight of this term. This term expands the margin of the data in the input space and the network needs to avoid the mapping of the margin to the origin in the pre-phase layer.

5.3. Training configuration

The neural network contains 5 hidden layers with 30 nodes in each layer ($Q = 5$, $H = 30$). Other parameters are $r = 3$, $w = 1$, $a = -2\pi/180$, $b = 15\pi/180$, $c = -0.75\pi$, $\alpha = 0.02$, $\gamma = 0.02$, $\sigma = 1$, $\lambda = 0.2$, and $\sigma_w = 0.5$. The training is run for 5000 iterations.

Unlike other experiments, the parameter a is set to a small negative value. This is because those repetitive movements of humans usually contain short pauses that have phase progression around zero or small negative values.

5.4. Result: Training data distribution in pre-phase layer

In all three movements, the networks are able to find a transformation that maps the training data from the input layer to a roughly circular distribution in the pre-phase layer as shown in Fig. 14.

5.5. Result: Phase extraction from unseen data

The trained networks are tested with the movements from the unseen subjects. Some examples of phase extraction results are shown in Fig. 15. The output phase sequences are responsive to the change in the speed of the movement despite the sudden changes. The extraction is also sensitive to short pauses which can be seen in all three movements where the phase sequence becomes flat for a short period of time.

6. Discussions

6.1. Flipped mapping problem

When parameter a is set to a negative value, the network can occasionally converge to the mapping where all the training data progress clockwise (backward) instead of counterclockwise. When this situation occurs, the network is usually trapped forever in a local minimum. A way to fix this problem is to repeat the same training multiple times with different weight initialization until the average phase progression becomes positive.

6.2. Constant phase offset (optional)

Because the proposed cost function (Eq. (2)) is rotational invariant at the pre-phase layer and the output layer, the mapped data distribution on those layers can be rotated around the origin without affecting the cost function. This fact allows the user to choose a particular spot on the waveform to be mapped to the transition point between two consecutive periods (i.e., the phase of 2π and zero). For example, the user may need the R peak of the ECG signal to be the beginning of each round for an easier interpretation. This can be done by adding a constant rotation after the output layer of the trained network. The amount of rotation can be retrieved by feed-forwarding the signal from the target spot into the trained network to get a phase. The negative of that phase value is used in the additional rotation.

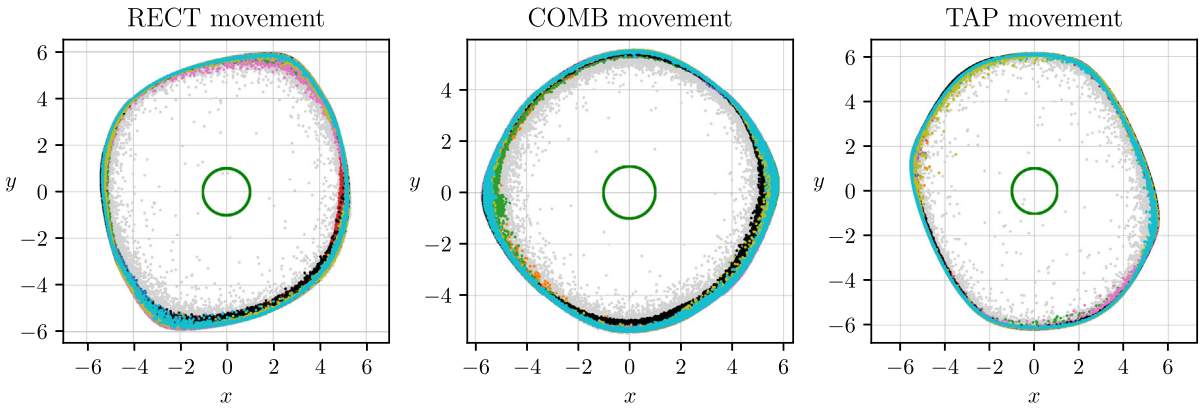


Fig. 14. Mapping results from RECT, COMB, and TAP movements after 5000 iterations of training. The light gray points are mapped from the noise-added input used in the *marginal singularity penalty* term. The green unit circle at the center is for a referencing purpose. The rest of the colored points are training data distribution in the pre-phase layer. Each color represents data from one subject. Proceed to the supplementary videos for their animations from every iteration.

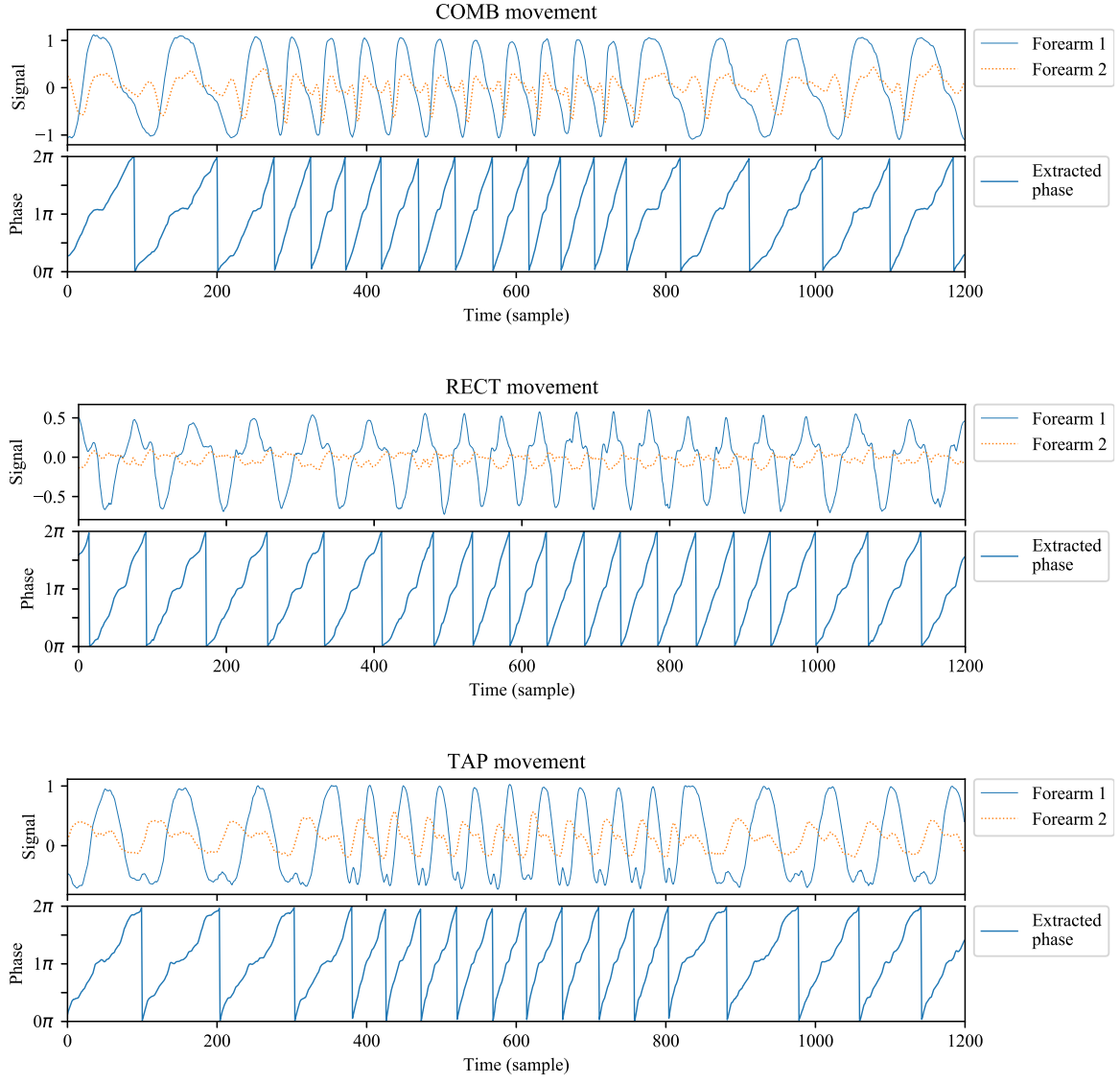


Fig. 15. Phase extractions of three unseen exercise data (COMB, RECT, and TAP). Forearm 1 and 2 are two channels taken from 11 channels of the movement record for a referencing purpose. The differences in the movement speed and short pauses can be observed in the phase sequences.

6.3. Potential applications

In the case of one-dimensional data such as ECG, if the signal is not similar to a sine wave, the Hilbert transform may not produce a good phase extraction. The proposed method can be used to extend the capability of Hilbert transform for better phase extraction.

In the case of multi-dimensional data such as kinematic features of human movement, the method is able to utilize all the dimensions automatically to generate one phase sequence. Such a task is not trivial even with a specific handcrafted procedure. Moreover, the method is able to deal with the variation of data from different subject anatomy and a range of movement speeds in both training and testing data.

With the extracted continuous phase data at every sample, all the recorded periods can be automatically segmented and time-warped into a standard length (from 0 to 2π) for phase-dependent statistical analysis. This way of usage is particularly useful for human movement analysis in physical exercises and rehabilitative assessments.

7. Conclusions

We have found that a neural network can learn the continuous phase extraction task from examples of a multidimensional signal without supervision. Being unsupervised allows the user to skip the tedious and inaccurate manual phase labeling process while maintaining the phase extraction quality above other generic methods such as the Hilbert transform or the complex wavelet convolution in terms of accuracy and shorter length of the input window. The flexibility of the method has been tested successfully in three different types of quasiperiodic time-series data — the synthetic 2D signals, the ECG signal, and 11D kinematic of human movements. While the traditional methods are limited to a single dimension, the proposed method can take advantage of high-dimensional input, resulting in the simultaneous improvements of 63% in accuracy and over 90% in the window size requirement. Because phase is one of the most fundamental properties of any periodic data, the method has the potential to be used in a considerably wide range of applications.

CRediT authorship contribution statement

Prayook Jatesiktat: Conceptualization, Methodology, Software, Formal analysis, Investigation, Visualization, Writing - original draft. **Guan Ming Lim:** Investigation, Validation, Writing - review & editing. **Christopher Wee Keong Kuah:** Investigation, Resources, Data curation, Funding acquisition. **Wei Tech Ang:** Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the second Rehabilitation Research Grant (ID: RRG2/16001) from Rehabilitation Research Institute of Singapore (RRIS). RRIS is a joint research institute by Nanyang Technological University (NTU), Agency for Science, Technology and Research (A*STAR), and National Healthcare Group (NHG). The majority of the materials contained in this article were previously published in Chapter 5 of the first author's Ph.D. thesis [18].

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.asoc.2020.106386.

References

[1] A. Murgia, P. Kyberd, T. Barnhill, The use of kinematic and parametric information to highlight lack of movement and compensation in the upper extremities during activities of daily living, Gait Posture 31 (3) (2010) 300–306, http://dx.doi.org/10.1016/j.gaitpost.2009.11.007.

[2] J.W. Lee, K.S. Kim, B. Lee, B. Lee, M.H. Lee, A real time qrs detection using delay-coordinate mapping for the microcontroller implementation, Ann. Biomed. Eng. 30 (9) (2002) 1140–1151, http://dx.doi.org/10.1114/1.1523030.

[3] D. Holden, T. Komura, J. Saito, Phase-functioned neural networks for character control, ACM Trans. Graph. 36 (4) (2017) 42:1–42:13, http://dx.doi.org/10.1145/3072959.3073663.

[4] S. Patel, R. Hughes, T. Hester, J. Stein, M. Akay, J.G. Dy, P. Bonato, A novel approach to monitor rehabilitation outcomes in stroke survivors using wearable technology, Proc. IEEE 98 (3) (2010) 450–461, http://dx.doi.org/10.1109/JPROC.2009.2038727.

[5] K. Kim, W.K. Song, J. Lee, H.Y. Lee, D.S. Park, B.W. Ko, J. Kim, Kinematic analysis of upper extremity movement during drinking in hemiplegic subjects, Clin. Biomech. 29 (3) (2014) 248–256, http://dx.doi.org/10.1016/j.clinbiomech.2013.12.013.

[6] S.B. Thies, P.A. Tresadern, L.P. Kenney, J. Smith, D. Howard, J.Y. Goulermas, C. Smith, J. Rigby, Movement variability in stroke patients and controls performing two upper limb functional tasks: a new assessment methodology, J. NeuroEng. Rehabil. 6 (1) (2009) 2, http://dx.doi.org/10.1186/1743-0003-6-2.

[7] S.L. Hahn, Hilbert Transforms in Signal Processing, Artech House, 1996.

[8] A. Pikovsky, M. Rosenblum, J. Kurths, J. Kurths, Instantaneous phase and frequency of a signal, in: Synchronization: A Universal Concept in Nonlinear Sciences, Vol. 12, Cambridge university press, 2003, pp. 362–370, Ch. A2.

[9] J.-P. Lachaux, E. Rodriguez, J. Martinerie, F.J. Varela, Measuring phase synchrony in brain signals, Hum. Brain Mapp. 8 (4) (1999) 194–208, http://dx.doi.org/10.1002/(SICI)1097-0193(1999)8:4<194::AID-HBM4>3.0.CO;2-C.

[10] R. Quian Quiroga, A. Kraskov, T. Kreuz, P. Grassberger, Performance of different synchronization measures in real data: A case study on electroencephalographic signals, Phys. Rev. E 65 (2002) 041903, http://dx.doi.org/10.1103/PhysRevE.65.041903.

[11] P. Jatesiktat, W.T. Ang, Unsupervised phase extraction using dual autoencoder, in: 2017 International Conference on Advanced Computing and Applications (ACOMP), 2017, pp. 71–76, http://dx.doi.org/10.1109/ACOMP.2017.15.

[12] P. Jatesiktat, D. Anopas, W.T. Ang, Unsupervised phase learning and extraction from repetitive movements, in: 2018 40th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC), 2018, pp. 227–230, http://dx.doi.org/10.1109/EMBC.2018.8512196.

[13] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G.S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I.J. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Józefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mane, R. Monga, S. Moore, D.G. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P.A. Tucker, V. Vanhoucke, V. Vasudevan, F.B. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, X. Zheng, Tensorflow: Large-scale machine learning on heterogeneous distributed systems, Comput. Res. Repos. (2016) (CoRR) abs/1603.04467. URL http://arxiv.org/abs/1603.04467.

[14] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, Comput. Res. Repos. (2014) (CoRR) abs/1412.6980. arXiv:1412.6980. URL http://arxiv.org/abs/1412.6980.

[15] P. Plawiak, ECG signals (1000 fragments), in: Mendeley Data, V3, 2017, http://dx.doi.org/10.17632/7dybx7wyfn.3.

[16] L. Marple, Computing the discrete-time "analytic" signal via fft, IEEE Trans. Signal Process. 47 (9) (1999) 2600–2603, http://dx.doi.org/10.1109/78.782222.

[17] P. Jatesiktat, D. Anopas, W.T. Ang, Personalized markerless upper-body tracking with a depth camera and wrist-worn inertial measurement units, in: 2018 40th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC), 2018, pp. 1–6, http://dx.doi.org/10.1109/EMBC.2018.8513068.

[18] P. Jatesiktat, Data-Driven Phase Extraction for Anomaly Detection of Repetitive Human Movements (Ph.D. thesis), Nanyang Technological University, 2019, http://dx.doi.org/10.32657/10220/47998.



Prayook Jatesiktat received the B.E. degrees in Computer Engineering from Kasetsart University, Thailand, in 2012 and the Ph.D. degree from the School of Mechanical and Aerospace Engineering, Nanyang Technological University, Singapore, in 2019. His research interests include unsupervised machine learning and machine perception.



Guan Ming Lim received the B.E. degrees in Mechanical and Aerospace Engineering from Nanyang Technological University, Singapore, in 2014. He is currently a Ph.D. candidate in the School of Mechanical and Aerospace Engineering, Nanyang Technological University. His research interests include robotics and machine learning.



Christopher Wee Keong Kuah is a Principal Occupational Therapist at Tan Tock Seng Hospital, Centre for Advanced Rehabilitation Therapeutics, Singapore. He received his professional qualification in 1995 from Nanyang Polytechnic of Singapore, attained the Bachelor of Health Science in Occupational Therapy from the University of Sydney in 1997, and was awarded the Master of Science in Neurorehabilitation from Brunel University in the United Kingdom in 2002. He is currently seconded to the Rehabilitation Research Institute of Singapore as a Research Scientist. His research scope includes applications of brain–computer interface, robotics, and sensor technologies for post-stroke movement recovery.



Wei Tech Ang received the B.E. and M.E. degrees in Mechanical and Production Engineering from Nanyang Technological University, Singapore, in 1997 and 1999, respectively, and the Ph.D. degree in robotics from Carnegie Mellon University, Pittsburgh, PA, USA, in 2004. Since 2004, he has been with the School of Mechanical and Aerospace Engineering, Nanyang Technological University, where he is currently an Associate Professor and the Executive Director of Rehabilitation Research Institute of Singapore. His research focuses on robotics technology for biomedical applications, which include surgery, rehabilitation, and cell micromanipulation.